

OICPC455

AY 2024-25

Machine Learning Lab

Quality Laboratory Manual

Experiment No. 11

Study and demonstrate a Simple Recommendation System



Course Instructor -
DR. TAHSEEN A. MULLA
ASSOCIATE PROFESSOR

Experiment No. 11

Title of Experiment: To study and demonstrate a simple recommendation system

Aim of Experiment: To implement and understand a basic recommendation system using collaborative filtering to suggest items to users based on their preferences

System Requirements – Win 8 and above OS, 4GB RAM, 2.33 GHz Processor

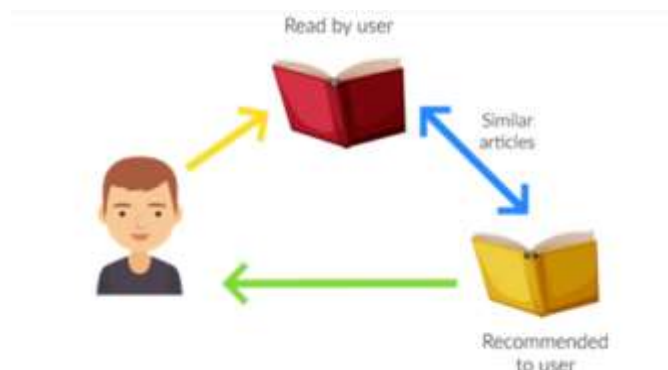
Software/s Needed for Experiment – Jupyter Notebook/ Anaconda Navigator/ Google Colaboratory/ Spyder, Python 3.x [With libraries such as Numpy, Pandas, Matplotlib, Scikit-Learn and Surprise for Collaborative Filtering]

Experiment Outcomes –

1. To understand the basics of recommendation system
2. To implement a user-based collaborative filtering model
3. To apply collaborative filtering to generate recommendations
4. To analyze and interpret recommendation results

Theory –

A recommendation system is a type of machine learning system that provides personalized recommendations to users based on their past behaviors, preferences, and patterns. It is a subclass of information filtering systems that use algorithms to recommend items to users based on their interests or behaviors



Recommendation systems are widely used in e-commerce, social media, entertainment, and other online platforms to increase user engagement and retention, improve customer satisfaction, and drive sales and revenue

How Recommendation System works?

- **Collecting user data:** The first step in building a recommendation system is to collect user data. This can include user ratings, reviews, clickstream data, purchase history, and other behavioral data. The data can be collected either explicitly, through user surveys or feedback forms, or implicitly, through user interactions with the platform
- **Storing the data:** Once the user data is collected, it needs to be stored in a database or data warehouse for analysis. The data can be stored in a structured or unstructured format, depending on the type and volume of the data
- **Analyzing the data:** The next step is to analyze the user data to identify patterns and trends. This can be done using various data analysis techniques like clustering, classification, and regression. The goal is to understand the user's preferences, behaviors, and interests, and to use this information to make personalized recommendations
- **Filtering and recommending:** The final step is to filter the data and make recommendations to the user. This can be done using various recommendation algorithms, such as collaborative, content-based, and hybrid filtering. The algorithm uses the user data and the analysis results to generate a list of recommended items the user will likely be interested in. The recommendations are then presented to the user in a personalized way, such as through a recommendation widget, email, or push notification

Procedure to implement a Recommendation System:

1. Data Collection and Preparation:

- Use a sample dataset like the MovieLens dataset which contains user ratings for different movies
- Load the dataset and clean it, if necessary by removing the missing or duplicate values

2. Create a User-Item Matrix:

- Restructure the data into a matrix where each row represents a user and each column represents an item and each cell holds the user's rating for that item
- This matrix is typically sparse because not all users have rated all items

3. Calculate User Similarity:

- Use a similarity measure like cosine similarity to calculate the similarity between users based on their ratings
- Higher similarity values indicate users with more preferences

4. Generate Recommendation using Collaborative Filtering:

- For a given user, identify the most similar users based on similarity scores
- Recommend items that similar users have rated highly but the target user has not rated

5. Evaluate the Recommendations:

- Compare the recommended items with the actual items the user has rated highly to evaluate accuracy
- Use metrics such as precision, recall or Mean Absolute Error (MAE) if test data is available

Sample Code:

```
import numpy as np
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import mean_squared_error
from math import sqrt

# Step 1: Load and Prepare the Dataset
# Using a hypothetical ratings dataset with columns: user_id,
# movie_id, rating
data = {
    'user_id': [1, 1, 1, 2, 2, 2, 3, 3, 3],
    'movie_id': [101, 102, 103, 101, 102, 104, 102, 103, 104],
    'rating': [5, 3, 4, 4, 5, 2, 2, 4, 5]
}
df = pd.DataFrame(data)

# Create User-Item Matrix
user_item_matrix = df.pivot_table(index='user_id',
                                   columns='movie_id', values='rating').fillna(0)

# Step 2: Calculate User Similarity Matrix
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(user_similarity,
                                  index=user_item_matrix.index, columns=user_item_matrix.index)

# Step 3: Make Recommendations
def get_recommendations(user_id, num_recommendations=2):

    # Find similar users
    similar_users =
    user_similarity_df[user_id].sort_values(ascending=False)[1:]

    # Find movies rated by similar users but not rated by the target
    user
    user_ratings = user_item_matrix.loc[user_id]
    unrated_movies = user_ratings[user_ratings == 0].index
```

```
movie_scores = {}

for similar_user in similar_users.index:
    similar_user_ratings = user_item_matrix.loc[similar_user]

    for movie in unrated_movies:
        if similar_user_ratings[movie] > 0:
            if movie not in movie_scores:
                movie_scores[movie] = 0
                # Weight the similar user's rating by similarity score
                movie_scores[movie] += similar_user_ratings[movie] *
similar_users[similar_user]

# Sort movies by score
recommended_movies = sorted(movie_scores.items(), key=lambda x:
x[1], reverse=True)

# Return the top recommendations
return [movie for movie, score in
recommended_movies[:num_recommendations]]

# Get recommendations for a user
user_id = 1
recommendations = get_recommendations(user_id)
print(f"Recommendations for User {user_id}: {recommendations}")
```

Observations -

- Track how similarity between users affects recommendations
- Evaluate the recommendations to see if they align with the target user's actual preferences

Conclusion –

Hence, the recommendation system summarizes the effectiveness of collaborative filtering in generating relevant recommendations and discuss potential improvements such as incorporating item-based collaborative filtering or content-based methods for enhanced performance

References –**a. Textbook –**

- i. Machine Learning with Python – An approach to Applied ML – Abhishek Vijayvargiya, BPB Publications, 1st Edition 2018
- ii. Machine Learning, Tom Mitchell, McGraw Hill Education, 1st Edition 1997

b. Online references –

- i. <https://www.datacamp.com/tutorial/recommender-systems-python>
- ii. <https://365datascience.com/tutorials/how-to-build-recommendation-system-in-python/>
- iii. <https://towardsdatascience.com/recommendation-system-series-part-1-an-executive-guide-to-building-recommendation-system-608f83e2630a>

Expected Oral Questions –

1. What is recommendation system and why is it useful?
2. Describe the main types of recommendation systems
3. What is collaborative filtering and how does it work?
4. Explain the concept of content-based filtering in recommendation systems
5. Explain how user-based collaborative filtering differs from item-based collaborative filtering
6. What is the cosine similarity metric and how is it used in recommendation systems?
7. Describe the matrix factorization technique in collaborative filtering
8. How does the scalability of recommendation systems impact performance in large datasets?
9. How are recommendation systems used in social media platforms?
10. How do location-based recommendation systems work?

FAQ's in Interview –

1. What is a recommendation system and how does it work?
2. Can you explain the difference between collaborative filtering and content-based recommendations?
3. What are the main challenges in building recommendation systems?
4. How do matrix factorization techniques work in recommendation engines?
5. What are the different evaluation metrics for recommender systems?